

Multi-Tenant E-commerce Application

User Stories — Authentication, Tenant Management & Core E-commerce

This document defines an incremental set of user stories for a multi-tenant e-commerce API. The proposed architecture uses FastAPI, Pydantic, SQLAlchemy and PostgreSQL, with tenant-aware authentication and authorization.

Epic 1 — Tenant Management

US-001 — Create a Tenant

As a merchant, I want to create a tenant/store so that I can operate my own ecommerce store.

Acceptance Criteria

- Tenant has a unique ID and slug.
- Tenant has a name and optional domain.
- Tenant has a status such as active/inactive.
- created_at and updated_at are maintained.
- Duplicate tenant slugs are rejected.

US-002 — Retrieve Tenant

As a tenant administrator, I want to retrieve my tenant information so that I can view my store details.

Acceptance Criteria

- Authenticated users can retrieve tenant information.
- User must belong to the requested tenant.
- Users from another tenant cannot access it.
- Inactive tenants cannot be used for normal operations.

Epic 2 — User Signup

US-003 — Register a User

As a customer, I want to create an account so that I can use the ecommerce application.

Acceptance Criteria

- Email is validated.
- Password is securely hashed and never stored in plain text.
- User is associated with the requested tenant.
- Email verification is initiated.
- Password is never returned in the API response.

US-004 — Verify Email

As a registered user, I want to verify my email so that my account can be activated.

Acceptance Criteria

- Verification token is sent to the user.
- Token has an expiration time.
- Valid token verifies the email.
- Expired or already-used tokens are rejected.
- Token cannot be reused.

US-005 — Resend Verification Email

As a registered user, I want to request another verification email if I did not receive the first one.

Acceptance Criteria

- A new token can be issued for an unverified account.
- Previous token becomes invalid.
- New token has an expiration time.
- Responses should avoid unnecessary account enumeration.

Epic 3 — Login

US-006 — Login to Tenant

As a user, I want to log in to a specific tenant so that I can access my store account.

Acceptance Criteria

- User provides tenant, email and password.
- Credentials are validated.
- User must belong to the specified tenant.
- User and tenant must be active.
- Authentication tokens are returned.
- Invalid credentials do not reveal which credential was incorrect.
- Login attempts are protected against abuse/rate limiting.

Epic 4 — Token Management

US-007 — Issue Access Token

As an authenticated user, I want an access token so that I can access protected APIs.

Acceptance Criteria

- Token identifies the user and tenant context.
- Token has an expiration time.
- Protected APIs reject missing or invalid tokens.
- Token does not contain sensitive information.

US-008 — Refresh Access Token

As an authenticated user, I want to refresh my access token without logging in again.

Acceptance Criteria

- Valid refresh token produces a new access token.
- Expired or revoked refresh tokens are rejected.
- Refresh tokens are associated with the correct user and tenant.
- Refresh-token rotation can be implemented.

US-009 — Logout

As a user, I want to log out so that my authentication session is terminated.

Acceptance Criteria

- Current authentication session/token is revoked.
- Revoked refresh tokens cannot be reused.
- Other sessions may remain active unless logout-everywhere is implemented.

Epic 5 — Password Management

US-010 — Forgot Password

As a user, I want to request a password reset when I forget my password.

Acceptance Criteria

- User submits an email.
- A password reset token is generated.
- Token has an expiration time.
- Reset instructions are sent.
- API should avoid revealing whether an email exists.

US-011 — Reset Password

As a user, I want to reset my password using a valid reset token.

Acceptance Criteria

- Valid token permits a password reset.
- Expired or used tokens are rejected.
- New password meets the password policy.
- Password is securely hashed.
- Existing sessions can be revoked after reset.

Epic 6 — User Profile

US-012 — View Current User

As an authenticated user, I want to view my profile.

Acceptance Criteria

- Returns current user information.
- Does not return password hash.
- Can return tenant memberships/roles where appropriate.
- Requires authentication.

US-013 — Update Profile

As an authenticated user, I want to update my profile information.

Acceptance Criteria

- Only permitted profile fields can be changed.
- User cannot change their authorization role through this endpoint.
- User cannot arbitrarily change tenant membership.
- Changes are persisted.

Epic 7 — Multi-Tenant Membership

US-014 — Associate User With Tenant

As a tenant administrator, I want to add a user to my tenant so that the user can access the store.

Acceptance Criteria

- Membership has a role.
- Membership can be active/inactive.
- Duplicate memberships are prevented.
- Only authorized tenant administrators can manage memberships.

US-015 — User Belongs to Multiple Tenants

As a user, I want to belong to multiple tenants so that I can access different stores using one account.

Acceptance Criteria

- One user can have multiple tenant memberships.
- Each membership can have a different role.
- Authentication establishes the active tenant.
- Authorization uses both user_id and tenant_id.

Epic 8 — Addresses

US-016 — Add Address

As a customer, I want to save my shipping/billing address so that I do not need to enter it for every order.

Acceptance Criteria

- Address belongs to a user.
- Address is associated with the appropriate tenant context.
- Address has a shipping/billing type.
- User can mark an address as default.
- User cannot access another user's addresses.

US-017 — Manage Addresses

As a customer, I want to update and delete my saved addresses.

Acceptance Criteria

- User can only access their own addresses.
- Deleted addresses cannot be used for future orders.
- Default-address rules are enforced.

Epic 9 — Tenant Isolation

US-018 — Prevent Cross-Tenant Data Access

As a platform owner, I want tenant data to be isolated so that one tenant cannot access another tenant's data.

Acceptance Criteria

- Tenant context is established for every protected request.
- Queries are scoped to tenant_id.
- Users cannot access another tenant's products, orders or categories.
- Changing an ID in a URL cannot bypass tenant isolation.

US-019 — Enforce Tenant Authorization

As a platform owner, I want authorization to consider both tenant membership and role.

Acceptance Criteria

- Authenticate user.
- Identify tenant.
- Check tenant membership.
- Check role.
- Check resource ownership where applicable.
- Allow or reject the request.

Epic 10 — Core E-commerce

US-020 — Manage Categories

As a tenant administrator, I want to create and manage product categories.

Acceptance Criteria

- Categories are tenant-specific.
- CRUD operations require appropriate authorization.

US-021 — Manage Products

As a seller/admin, I want to create and manage products.

Acceptance Criteria

- Products belong to a tenant.
- CRUD operations require appropriate authorization.
- Product access is tenant-scoped.

US-022 — Place Order

As a customer, I want to place an order for products.

Acceptance Criteria

- Order belongs to the authenticated user and active tenant.
- Products must belong to the same tenant.
- Product prices are captured at order creation.
- Order items contain quantity and unit price.
- Shipping address is captured.
- Order total is calculated server-side.
- Client cannot manipulate the order total.

Recommended API Endpoints

Area	Method	Endpoint
Authentication	POST	/api/v1/auth/signup
Authentication	POST	/api/v1/auth/login
Authentication	POST	/api/v1/auth/logout
Authentication	POST	/api/v1/auth/refresh
Authentication	POST	/api/v1/auth/forgot-password
Authentication	POST	/api/v1/auth/reset-password
Authentication	POST	/api/v1/auth/verify-email
Authentication	POST	/api/v1/auth/resend-verification
Profile	GET	/api/v1/me
Profile	PATCH	/api/v1/me
Tenant	POST	/api/v1/tenants
Tenant	GET	/api/v1/tenants/{tenant_id}
Addresses	POST	/api/v1/me/addresses
Addresses	GET	/api/v1/me/addresses
Addresses	PATCH	/api/v1/me/addresses/{id}
Addresses	DELETE	/api/v1/me/addresses/{id}
Categories	CRUD	/api/v1/categories
Products	CRUD	/api/v1/products
Orders	POST	/api/v1/orders

Core Database Relationship

- Tenants 1:N TenantUsers N:1 Users
- Users 1:N UserAddresses
- Users 1:N AuthTokens
- Users 1:N PasswordResetTokens
- Users 1:N EmailVerificationTokens
- Users 1:N Orders
- Orders 1:N OrderItems N:1 Products
- Categories 1:N Products

Recommended Implementation Order

- Phase 1: PostgreSQL → SQLAlchemy models → Tenants → Users → TenantUsers
- Phase 2: Signup → password hashing → email verification
- Phase 3: Login → JWT access token → refresh token → logout
- Phase 4: Forgot password → reset password
- Phase 5: Authentication dependencies → tenant isolation → RBAC

- Phase 6: Products → Categories → Cart → Orders → Payments

Key architectural decision: model Users ↔ TenantUsers ↔ Tenants as separate concepts. This allows one user to belong to multiple stores and have a different role in each store.